

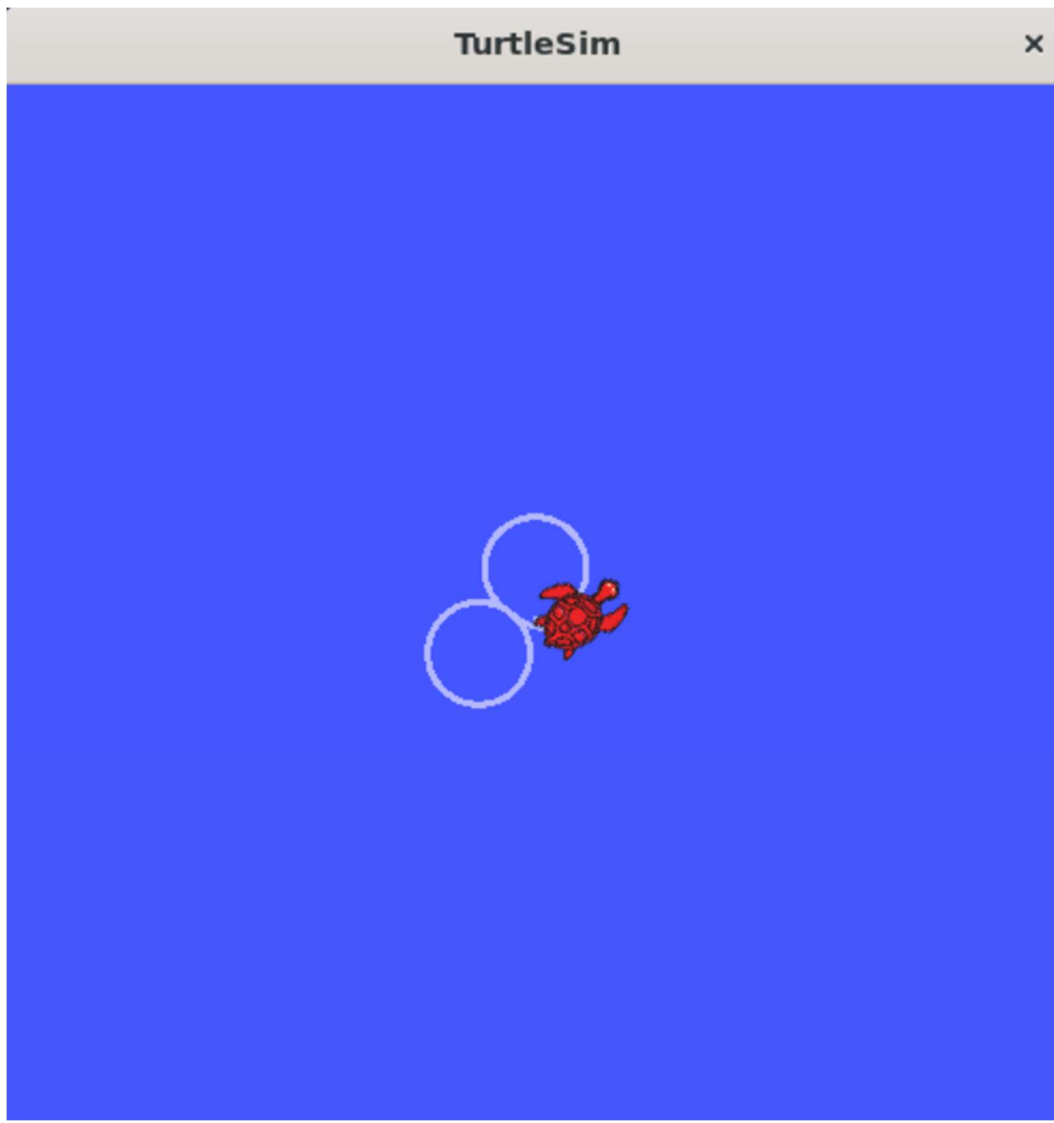
LAB 2: Basics of ROS II

By: Vy Tang

ID: 77659006

Objective

Problem 1: SwimNode



Code Explanation and Breakdown

```

1  #!/usr/bin/env python3
2
3  import rospy
4  from geometry_msgs.msg import Twist
5  import math
6  import random
7
8  class SwimNode:
9      def __init__(self):
10         # Initialize the ROS node
11         rospy.init_node('swim_node')
12         self.pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
13         self.rate = rospy.Rate(10) #10 Hz
14
15         # Initialize with Gaussian random velocities (positive only)
16         self.linear_velocity = abs(random.gauss(1.5, 0.5))
17         self.angular_velocity = abs(random.gauss(1.5, 0.5))
18
19         # Direction control variables
20         self.direction = 1
21         self.switch = 2 * (math.pi / self.angular_velocity)
22         self.last_switch_time = rospy.Time.now()
23
24     def motion_eight(self):
25         vel_msg = Twist()
26
27         while not rospy.is_shutdown():
28             current_time = rospy.Time.now()
29             elapsed = (current_time - self.last_switch_time).to_sec()
30
31             # Switch direction when interval elapsed
32             if elapsed >= self.switch - 0.01: # small buffer
33                 self.direction *= -1
34                 self.last_switch_time = current_time
35
36             # Set velocities
37             vel_msg.linear.x = self.linear_velocity
38             vel_msg.angular.z = self.direction * self.angular_velocity
39
40             self.pub.publish(vel_msg)
41             self.rate.sleep()
42
43 if __name__ == '__main__':
44     try:
45         node = SwimNode()
46         node.motion_eight()
47     except rospy.ROSInterruptException:
48         pass

```

For the first problem, I created a class and initialized the nodes using the `init_node` and `.publisher` to initialize my nodes and declare the publisher from the `rospy` library. Then I set a random arbitrary value using the `random` library and set the forward and turning speed.

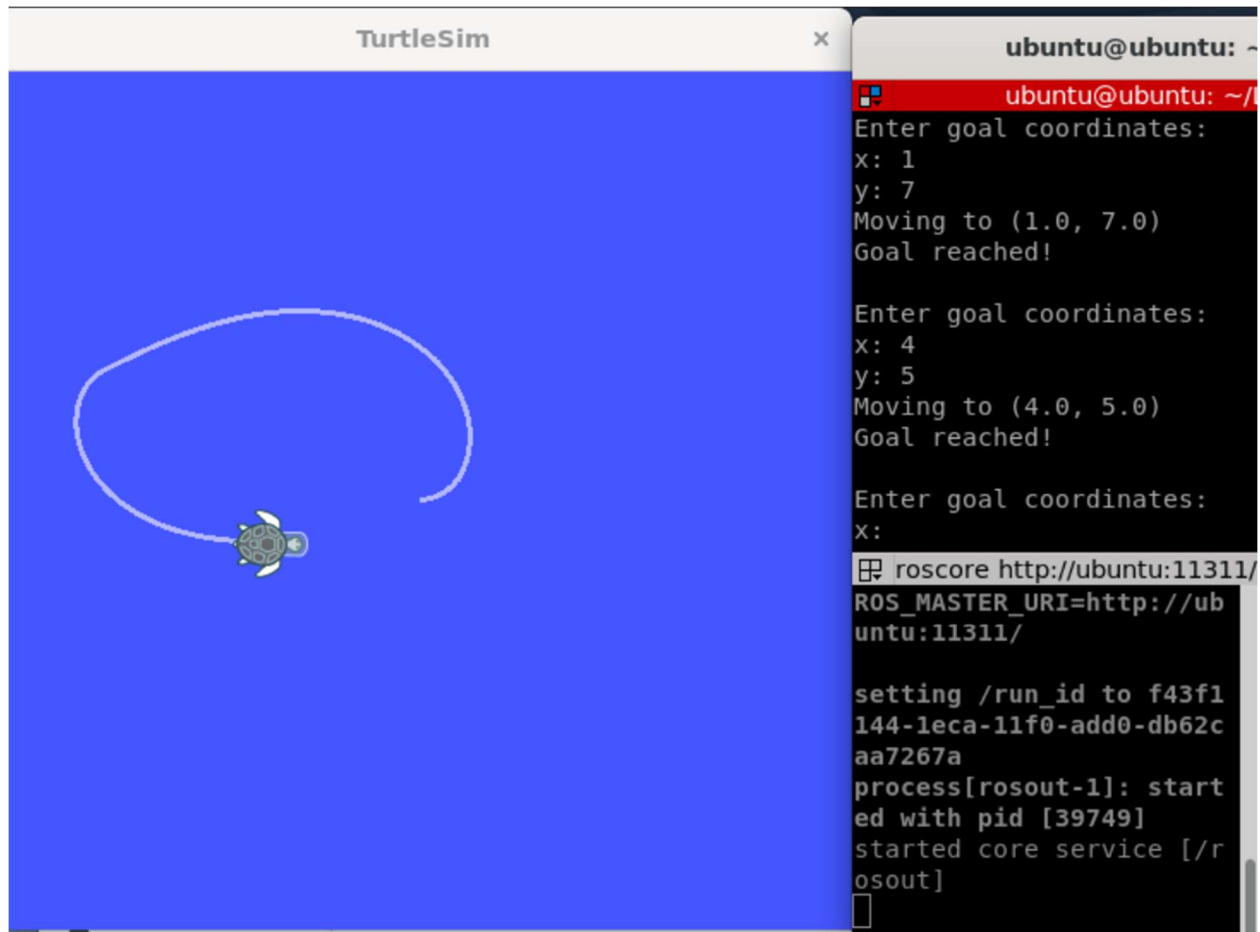
Next, is the direction control setup. Here, the direction is initially set to 1 which is positive for left turns and it will alternate between 1 and -1 for left and right turns. The switch method is to indicate the period in which the turtle will switch and for this, we will use the `math` library to get the pi value as the period is $T = 2\pi/w$ where w is the angular velocity set previously. After that, keep track of the last time there was a change in direction using the `.time` method.

For the function responsible for creating the figure 8, the `Twist` message for velocity commands is called and inside the while loops which keeps running until ROS shuts down, it tracks the elapsed time since the last direction change. Then, an if statement will check if the elapsed time is greater than the switch interval (period) and if it does, flip the direction and update the last switched time to the current time. After that, I set the constant forward speed and the turning speed in the twist message and

publishes the velocity command to the turtle. The final code block with an if-statement is for when the program ends to stop the curses function. It will also check if there is any launch failures.

The final code block handles main execution and creates node instance and starts motion as well as handling interruption cases.

Problem 2: SwimtoNode



Code Explanation and Breakdown

```
1  #!/usr/bin/env python3
2
3  import rospy
4  import math
5  from geometry_msgs.msg import Twist
6  from turtlesim.msg import Pose
7
8  class SwimToGoal:
9      def __init__(self):
10         # Initialize the ROS node
11         rospy.init_node('swim_to_goal')
12         self.pub = rospy.Publisher('/turtle1/cmd_vel', Twist, queue_size=10)
13         self.pose_sub = rospy.Subscriber('/turtle1/pose', Pose, self.pose_callback)
14         self.rate = rospy.Rate(10) #10 Hz
15
16         # Control parameters
17         self.K_linear = 1.5 # Proportional constant for linear velocity
18         self.K_angular = 4.0 # Proportional constant for angular velocity
19         self.position_tolerance = 0.5 # Distance threshold to consider goal reached
20
21         # Turtle state
22         self.current_pose = None
23         self.goal_x = None
24         self.goal_y = None
25
26     def pose_callback(self, data):
27         """Update current turtle position"""
28         self.current_pose = data
29
30     def get_user_input(self):
31         """Get goal coordinates from user"""
32         print("\nEnter goal coordinates:")
33         self.goal_x = float(input("x: "))
34         self.goal_y = float(input("y: "))
35         print(f"Moving to ({self.goal_x}, {self.goal_y})")
36
37     def calculate_errors(self):
38         """Compute position and angle errors"""
39         if self.current_pose is None:
40             return None, None
41
42         # Position error (Euclidean distance)
43         error_x = self.goal_x - self.current_pose.x
44         error_y = self.goal_y - self.current_pose.y
45         position_error = math.sqrt(error_x**2 + error_y**2)
```

```

37     def calculate_errors(self):
46
47         # Angle error (difference between current and desired heading)
48         desired_angle = math.atan2(error_y, error_x)
49         angle_error = desired_angle - self.current_pose.theta
50
51         # Normalize angle error to  $[-\pi, \pi]$ 
52         angle_error = math.atan2(math.sin(angle_error), math.cos(angle_error))
53
54         return position_error, angle_error
55
56     def control_loop(self):
57         rate = rospy.Rate(10) # 10Hz
58
59         while not rospy.is_shutdown():
60             if self.current_pose is None:
61                 rate.sleep()
62                 continue
63
64             # Get new goal if none set
65             if self.goal_x is None or self.goal_y is None:
66                 self.get_user_input()
67                 continue
68
69             # Calculate errors
70             pos_err, ang_err = self.calculate_errors()
71
72             # Check if goal reached
73             if pos_err < self.position_tolerance:
74                 print("Goal reached!")
75                 self.goal_x = None
76                 self.goal_y = None
77                 self.stop_turtle()
78                 continue
79
80             # Calculate and publish velocities
81             cmd = Twist()
82             cmd.linear.x = self.K_linear * pos_err
83             cmd.angular.z = self.K_angular * ang_err
84             self.pub.publish(cmd)
85
86             rate.sleep()

```

```

88     def stop_turtle(self):
89         """Stop the turtle's movement"""
90         cmd = Twist()
91         cmd.linear.x = 0
92         cmd.angular.z = 0
93         self.pub.publish(cmd)
94
95     if __name__ == '__main__':
96         try:
97             controller = SwimToGoal()
98             controller.control_loop()
99         except rospy.ROSInterruptException:
100            pass

```

The approach to initialize the publisher is still relatively the same except this time, there will be a subscriber to the publisher turtle1 which serves to read the turtle's current pose. To further initialize the node, I set the proportional gains for velocity control and the position tolerance of how close the turtle must get to its goal which is 0.5 units. Also, the turtle state is also defined to be in origin.

Next, I defined several function methods to help me with calculation. First, I created a pose_callback method to update the current pose whenever the new pose data arrives. There is also a get_user_input method to prompt the user for x and y coordinates of where the user wants the turtle to end up i.e. the goal_x, goal_y. The calculate_errors method computes the Euclidian distance to the goal (i.e. the position error) as well as calculating the angle difference between the current heading and goal direction. Finally, to normalize the angle error, I used the math.atan method call to normalize the angle error to $[-\pi, \pi]$.

After defining my necessary methods, I will finally implement the control loop function which is the core logic of the swim_to_goal node. First, there is a while loop that continuously loops and keeps running until ROS is shutdown. In the first two if-statements, the first one will check if there is pose data yet and if there isn't, it will delay the execution of the program until there is a pose data. The second if-statement will check if there is a goal yet and if there isn't it will set a new goal for the turtle. Then, we will actually call the calculate_errors method defined earlier to then check if the turtle has reached the goal within the tolerance distance (i.e. 0.5 as mentioned in the beginning). The turtle will then stop and the goals of the turtle will be reset once the turtle reaches its goal. Finally, there is the stop_turtle method which publishes a zero-velocity command to halt the turtle.

The final code block handles main execution and creates node instance and starts motion as well as handling interruption cases.

